



UNIVERSIDAD CATÓLICA DE SALTA
Licenciatura en Ciencia de Datos

Trabajo Práctico N° 3

Introducción a las Estructuras de Datos

Alumno: Facundo Luna

Profesor: Ing. Carlos Pastrana

Fecha: 10 de mayo de 2026

Índice

1. Indique cuál es el máximo número de nodos de nivel n en un árbol binario, y un árbol n -ario. 2
2. Indique cuál es la cantidad de árboles que se pueden formar con n nodos, y un nivel máximo m . 3
3. Indique como implementaría el juego ta-te-ti con el uso de la estructura de datos árbol. ¿Por qué se puede considerar que haya programas para jugar ta-te-ti de una forma perfecta, pero no para el juego de ajedrez o de damas? 5

1. Indique cuál es el máximo número de nodos de nivel n en un árbol binario, y un árbol n -ario.

Solución:

Árbol Binario

En un árbol binario, cada nodo tiene como máximo 2 hijos. El máximo número de nodos en el nivel n está dado por:

$$N_{\text{nivel}}(n) = 2^n$$

donde $n \geq 0$ representa el nivel del árbol (la raíz está en el nivel 0).

Ejemplos:

- Nivel 0: $2^0 = 1$ nodo
- Nivel 1: $2^1 = 2$ nodos
- Nivel 2: $2^2 = 4$ nodos
- Nivel 3: $2^3 = 8$ nodos

Árbol n -ario

En un árbol n -ario, cada nodo tiene como máximo n hijos, donde n es el grado del árbol. El máximo número de nodos en el nivel k está dado por:

$$N_{\text{nivel}}(k) = n^k$$

donde $k \geq 0$ representa el nivel del árbol y $n > 0$ es el número máximo de hijos por nodo.

Ejemplos para $n = 3$ (árbol ternario):

- Nivel 0: $3^0 = 1$ nodo
- Nivel 1: $3^1 = 3$ nodos
- Nivel 2: $3^2 = 9$ nodos
- Nivel 3: $3^3 = 27$ nodos

[↑ Volver al índice](#)

2. Indique cuál es la cantidad de árboles que se pueden formar con n nodos, y un nivel máximo m .

Solución:

Árboles Binarios con n Nodos (sin restricción de altura)

El número de árboles binarios distintos que se pueden formar con n nodos está dado por el **n -ésimo número de Catalan**:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)! \cdot n!}$$

Este número surge porque al construir un árbol binario con n nodos, debemos decidir cómo distribuir los nodos entre el subárbol izquierdo y el derecho. Si la raíz tiene i nodos en su subárbol izquierdo, entonces tendrá $n - 1 - i$ nodos en el derecho, y cada configuración se puede combinar de diferentes formas.

Ejemplos de los primeros valores:

- $n = 1$: $C_1 = 1$ árbol (solo la raíz)
- $n = 2$: $C_2 = 2$ árboles posibles
- $n = 3$: $C_3 = 5$ árboles posibles
- $n = 4$: $C_4 = 14$ árboles posibles
- $n = 5$: $C_5 = 42$ árboles posibles

Árboles Binarios con n Nodos y Altura Máxima m

Cuando se añade la restricción de altura máxima m , el número de árboles posibles disminuye, ya que se eliminan las configuraciones muy desbalanceadas (como los árboles que crecen solo hacia un lado).

No existe una fórmula simple para este caso, pero se puede calcular observando que:

- Cada subárbol (izquierdo y derecho) debe tener altura máxima $m - 1$
- Debemos distribuir los $n - 1$ nodos restantes (sin contar la raíz) entre ambos subárboles respetando esta restricción

Ejemplo ilustrativo con $n = 3$ nodos:

- Sin restricción de altura: $C_3 = 5$ árboles posibles
- Con altura máxima $m = 1$: Solo 1 árbol posible (raíz con dos hijos)
- Con altura máxima $m = 2$: 4 árboles posibles (se permiten algunas configuraciones desbalanceadas)

La restricción de altura actúa como un filtro que descarta los árboles que "crecen demasiado" en una dirección.

[↑ Volver al índice](#)

3. Indique como implementaría el juego ta-te-ti con el uso de la estructura de datos árbol. ¿Por qué se puede considerar que haya programas para jugar ta-te-ti de una forma perfecta, pero no para el juego de ajedrez o de damas?

Solución:

Implementación del Ta-Te-Ti con Árbol de Juego

El ta-te-ti se puede implementar mediante un **árbol de decisión**, donde cada nodo representa un estado completo del tablero.

Estructura del Nodo

Cada nodo del árbol contiene:

- **Estado del tablero:** Una matriz de 3×3 que representa las posiciones ocupadas por 'X', 'O' o vacías.
- **Coordenadas del último movimiento:** Par (i, j) que indica la posición del movimiento que generó este estado.
- **Turno actual:** Jugador que debe mover ('X' u 'O').
- **Evaluación del estado:** Indica si el estado es ganador, perdedor, empate o está en juego.
- **Lista de hijos:** Referencias a todos los estados alcanzables desde el estado actual mediante un movimiento legal.

Construcción del Árbol

1. **Raíz:** Representa el tablero vacío (estado inicial del juego).
2. **Nivel 1:** Contiene los 9 posibles estados resultantes de la primera jugada de 'X' (una jugada en cada casilla).
3. **Nivel 2:** Cada nodo del nivel 1 genera hasta 8 hijos (las casillas restantes donde 'O' puede jugar).
4. **Niveles subsiguientes:** Se continúa expandiendo el árbol alternando entre 'X' y 'O' hasta alcanzar estados terminales (victoria, derrota o empate).

Representación visual simplificada:

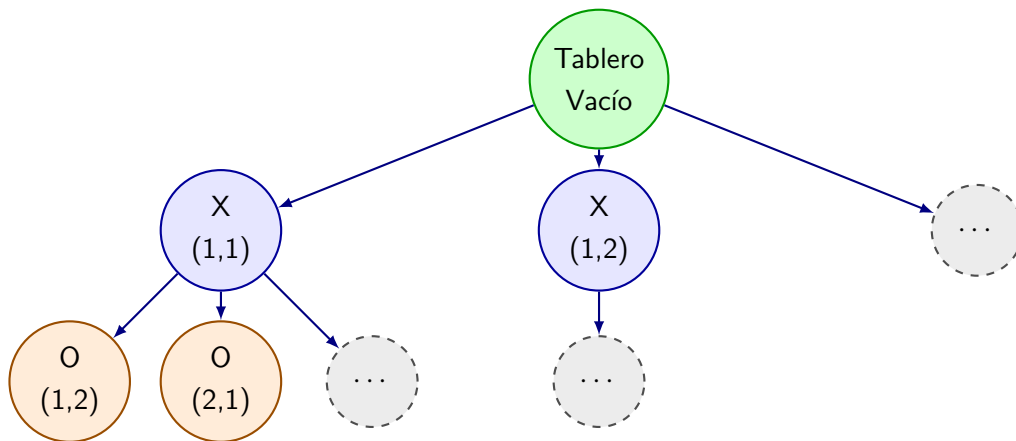


Figura 1: Representación parcial del árbol de juego del ta-te-ti. La raíz representa el tablero vacío, los nodos del primer nivel muestran las jugadas iniciales de 'X', y los del segundo nivel las respuestas de 'O'. Los puntos suspensivos indican ramas no mostradas por simplicidad.

Algoritmo Minimax

Para determinar la jugada óptima en cada estado, se utiliza el algoritmo **Minimax**:

- **Maximizador ('X'):** Busca maximizar su puntuación (victoria = +1, empate = 0, derrota = -1).
- **Minimizador ('O'):** Busca minimizar la puntuación de 'X' (equivalente a maximizar la suya propia).
- Se evalúan recursivamente todos los estados hijos y se propaga el valor óptimo hacia arriba.

¿Por qué el Ta-Te-Ti es Resoluble Perfectamente?

El ta-te-ti puede ser resuelto de forma perfecta porque:

1. Espacio de estados finito y pequeño:

- Máximo teórico: $9! = 362,880$ estados (considerando el orden de las jugadas).
- En la práctica: Aproximadamente 5,478 estados únicos (eliminando simetrías y estados imposibles).

2. Profundidad limitada:

- Máximo 9 movimientos (5 de 'X' y 4 de 'O').
- El árbol completo tiene altura máxima 9.

3. Exploración completa factible:

- Es computacionalmente viable explorar **todos** los estados posibles.
- Se puede precalcular el árbol completo y almacenarlo en memoria.

4. Determinismo total:

- No hay aleatoriedad ni información oculta.
- Dado un estado, se conocen exactamente todos los estados futuros posibles.

Resultado del análisis completo: Con juego perfecto de ambos jugadores, el ta-te-ti siempre termina en empate.

¿Por qué el Ajedrez y las Damas NO Son Resolubles Perfectamente?

Complejidad del Ajedrez

- **Número de Shannon:** Aproximadamente 10^{43} posiciones legales.
- **Árbol de juego:** Estimado en 10^{120} nodos (más que átomos en el universo observable: $\sim 10^{80}$).
- **Profundidad promedio:** Una partida típica dura 40 movimientos (80 medios movimientos), pero puede extenderse indefinidamente.
- **Factor de ramificación:** Promedio de 35 movimientos legales por posición.
- **Imposibilidad práctica:** Incluso con las computadoras más potentes, es **físicamente imposible** explorar todo el árbol de juego.

Complejidad de las Damas

- **Espacio de estados:** Aproximadamente 10^{20} posiciones posibles.
- **Resolución parcial:** En 2007, Jonathan Schaeffer y su equipo **resolvieron** las damas inglesas (8x8) usando 200 computadoras durante 18 años.
- **Resultado:** Con juego perfecto, las damas inglesas siempre terminan en **empate**.
- **Diferencia clave:** Aunque resuelto, requirió recursos computacionales masivos y décadas de trabajo, a diferencia del ta-te-ti que puede resolverse en milisegundos.

Comparación de Complejidad

Característica	Ta-Te-Ti	Damas	Ajedrez
Estados posibles	$\sim 5,500$	$\sim 10^{20}$	$\sim 10^{43}$
Árbol de juego	$\sim 10^5$	$\sim 10^{31}$	$\sim 10^{120}$
Profundidad máxima	9	~ 100	Ilimitada
¿Resoluble?	Sí (trivial)	Sí (costoso)	No (imposible)
Tiempo de resolución	Milisegundos	18 años	—

Cuadro 1: Comparación de complejidad entre juegos

Conclusión: La diferencia fundamental radica en la **complejidad combinatoria**. Mientras el ta-te-ti tiene un espacio de estados tan pequeño que permite su exploración completa, el ajedrez tiene un espacio tan vasto que supera cualquier capacidad computacional imaginable, haciendo imposible su resolución perfecta mediante fuerza bruta.

[↑ Volver al índice](#)